Technical Report:
.NET C# AI Code Analysis System

1 Introduction

This section introduces the motivation, objectives, and scope of the proposed system. With the increasing complexity of software systems, traditional code analysis methods are often insufficient for handling large-scale and security-critical applications.

2 System Overview

This section provides a high-level description of the system functionality and supported operations. The system is designed as a multi-purpose analysis tool that supports a wide range of software engineering and cybersecurity tasks. It allows users to interactively input code, configure analysis parameters, and receive structured outputs generated by AI models. The system aims to provide flexibility, scalability, and ease of use for both developers and researchers.

The system supports the following analysis types:

- Security analysis
- Reverse engineering
- Malware detection
- Code summarization
- Complexity analysis
- Code optimization

3 System Architecture

This section describes the architectural design and organization of system components. A modular and layered architecture is adopted to ensure separation of concerns, scalability, and maintainability. Each layer is responsible for a specific functionality, enabling independent development and integration of components. This approach also facilitates future extensions such as adding new models or analysis techniques.

The system consists of the following layers:

- **Presentation Layer**: Provides the graphical user interface (WinForms/WPF)
- **Application Layer**: Handles event-driven logic and user interactions
- **Integration Layer**: Manages communication with external AI services
- **Data Layer**: Processes structured data using JSON

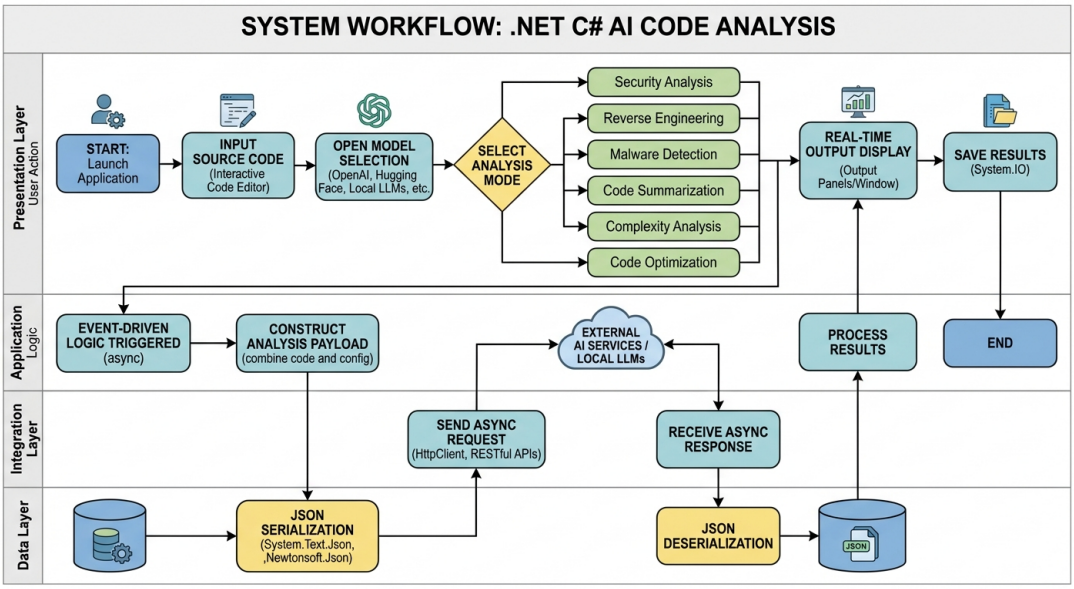

Figure 1: Sysem workflow

4 Key Features

This section highlights the primary functionalities that define the system capabilities. The system is designed to provide an intuitive and efficient interface for performing complex analysis tasks. It integrates multiple components into a unified environment, allowing seamless interaction between the user and AI models. These features contribute to improved usability and performance.

- Interactive code input editor
- Multiple analysis modes
- LLM provider selection
- Model and language configuration
- File handling (open/save)
- Multi-window interface
- Real-time output display

5 Technologies Used

This section outlines the technologies and tools used in the implementation of the system. The selection of technologies is based on performance, scalability, and compatibility with modern development practices. The system leverages both traditional software engineering tools and modern AI integration frameworks. This combination ensures robustness and extensibility.

5.1 Core Technologies

This subsection describes the foundational technologies used for development. These technologies provide the base environment for building and running the application. They support desktop application development and integration with external services. Their maturity and reliability make them suitable for production-level systems.

- C# (.NET Framework / .NET 6+)
- WinForms / WPF
- Visual Studio

5.2 API and Integration

This subsection focuses on communication mechanisms used to interact with external systems. The system relies on standard web-based protocols to send and receive data from AI services. Efficient serialization and deserialization techniques are used to handle structured data. These technologies enable seamless integration with various APIs.

- RESTful APIs
- HttpClient
- JSON (Newtonsoft.Json, System.Text.Json)

5.3 AI Integration

This subsection highlights the AI models and services integrated into the system. The system supports multiple providers to ensure flexibility and adaptability. These models enable advanced analysis capabilities such as reasoning, summarization, and pattern detection. The integration allows the system to leverage state-of-the-art AI techniques.

- OpenAI API
- Hugging Face API
- Azure OpenAI
- Local LLM models

6 .NET Techniques Applied

This section explains the key programming techniques used in the system implementation. These techniques ensure that the application is efficient, responsive, and maintainable. They also enable smooth interaction between the user interface and backend services. Modern .NET practices are applied to improve performance and scalability.

- Event-driven programming
- Asynchronous programming using `async/await`
- HTTP communication using `HttpClient`
- JSON serialization and deserialization
- Object-oriented programming (OOP)
- Modular UI design
- File handling using System.IO
- Menu-driven navigation
- Extensible architecture

7 System Interface

This section presents the graphical user interface and interaction design of the system. The interface is designed to be intuitive and user-friendly, allowing users to easily navigate and configure analysis tasks. It provides a structured layout that separates input, configuration, and output components. The design enhances usability and improves overall user experience.

7.1 Main Interface

This subsection presents the primary interface used for interacting with the system. It includes the main components such as code input, configuration settings, and output display. The layout is organized to provide clarity and efficiency in workflow. Users can easily perform analysis tasks from this interface.

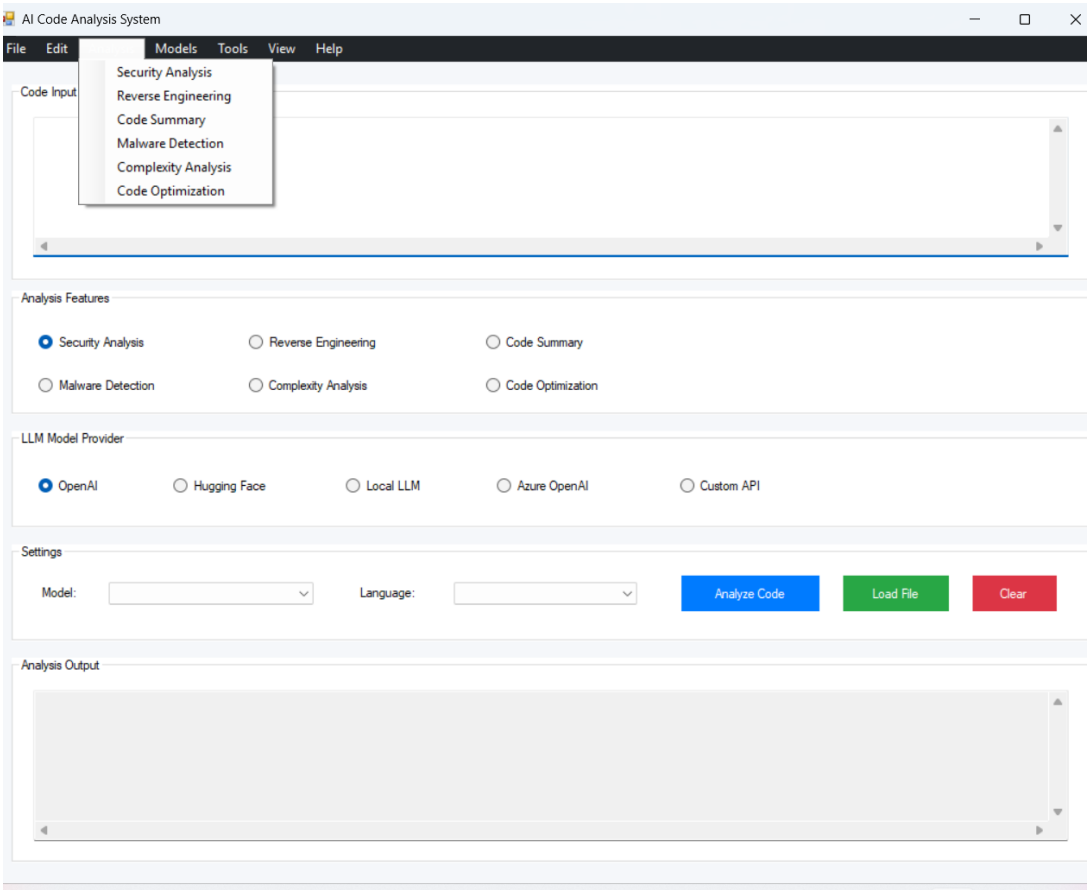

Figure 2: Main interface showing input, settings, and output panels

7.2 Analysis Window

This subsection describes the dedicated interface for performing detailed analysis. It allows users to configure advanced parameters such as analysis depth and language. The window provides additional flexibility for specialized tasks. It enhances the overall analytical capabilities of the system.

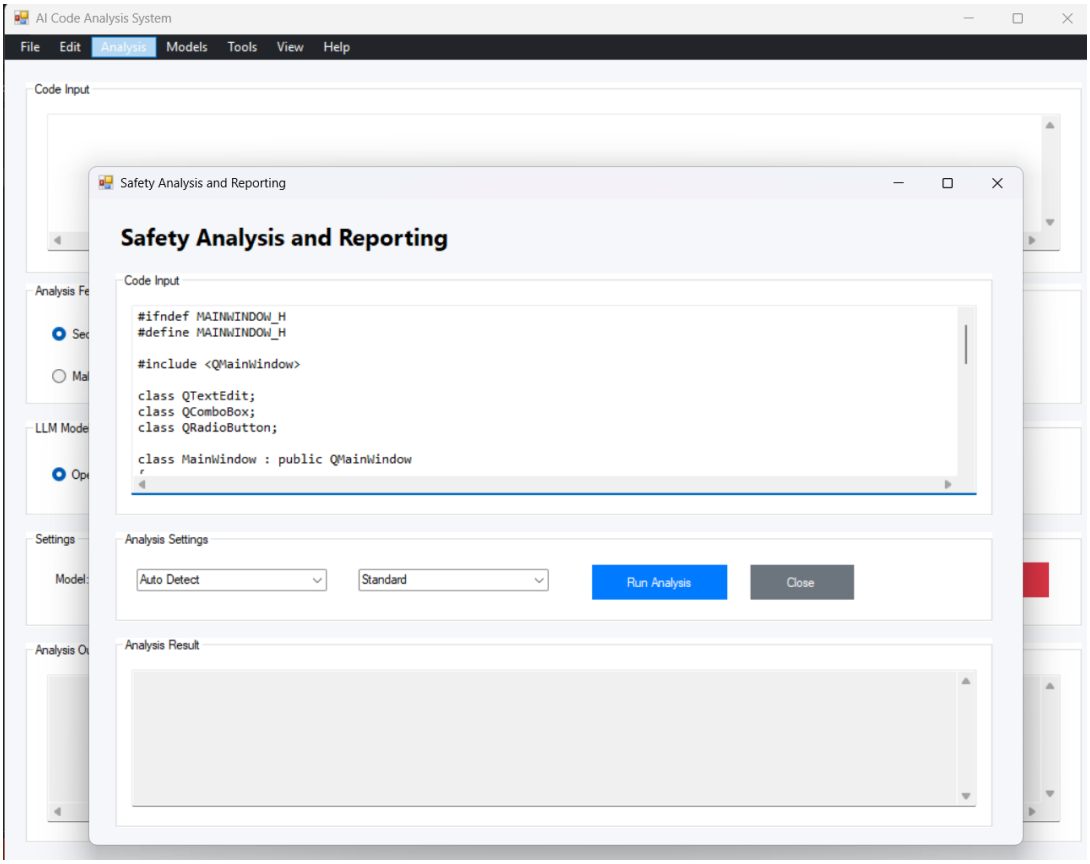

Figure 3: Analysis window with configurable parameters

7.3 Model Selection Interface

This subsection illustrates how users select and configure AI models. It provides options for choosing different providers and models. The interface supports both cloud-based and local LLM configurations. This flexibility allows users to adapt the system to their specific needs.

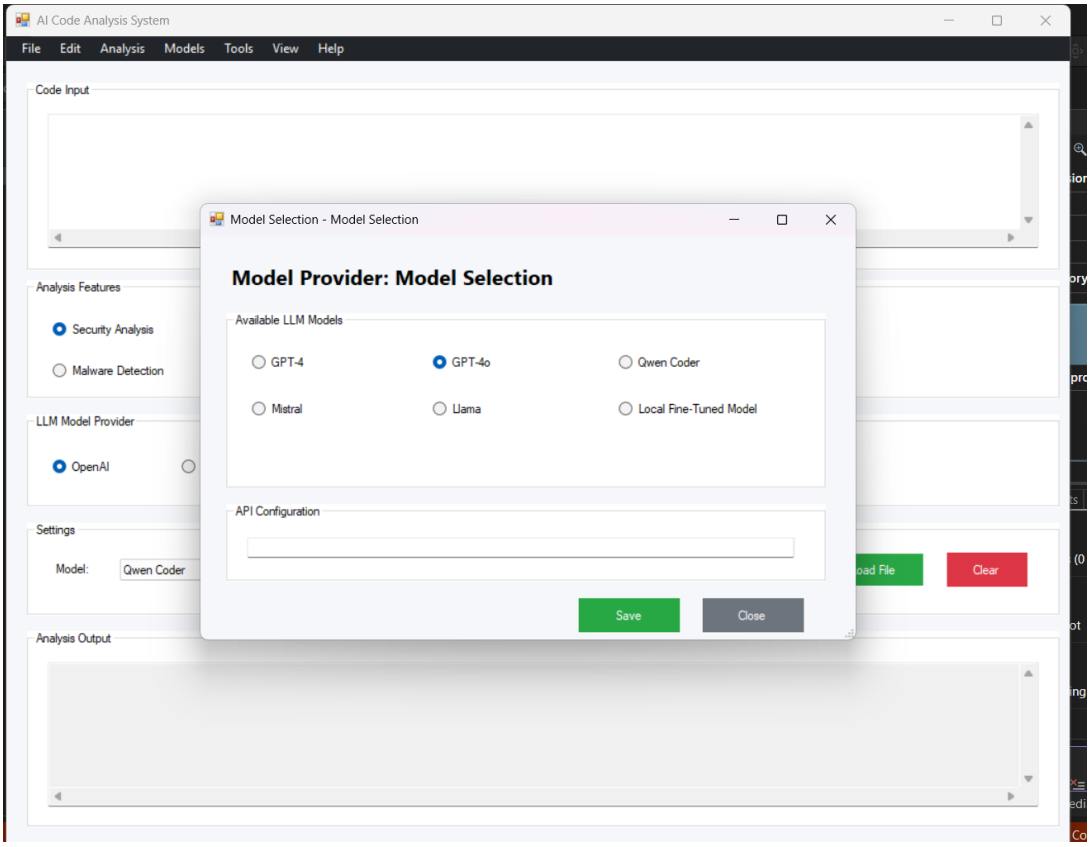

Figure 4: Model selection interface

8 Advantages

This section outlines the benefits and strengths of the system. The integration of AI and traditional software engineering provides a powerful tool for developers. The system is designed to be scalable, modular, and easy to use. It offers significant improvements in efficiency and analysis capability.

- AI-enhanced code analysis
- User-friendly interface
- Multi-model support
- Scalable and modular design
- Suitable for research and development